



Python Programming

WHAT IS PYTHON?

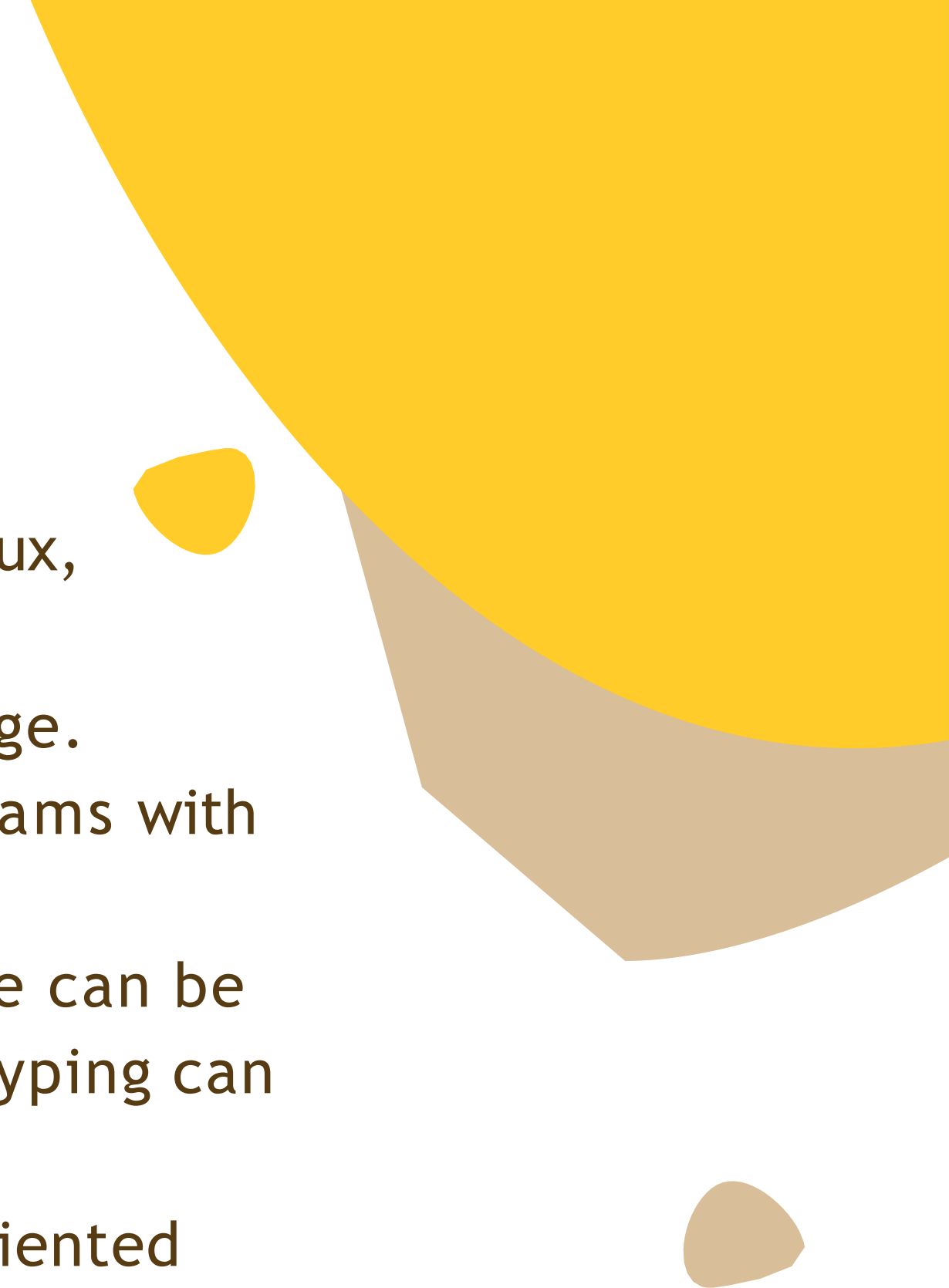
- Python is a popular programming language.
- It was created by **Guido van Rossum**, and released in **1991**.

It is used for:

- web development (server-side),
- software development,
- mathematics,
- system scripting.

WHY PYTHON?

- Python works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc).
- Python has a simple syntax similar to the English language.
- Python has syntax that allows developers to write programs with fewer lines than some other programming languages.
- Python runs on an interpreter system, meaning that code can be executed as soon as it is written. This means that prototyping can be very quick.
- Python can be treated in a procedural way, an object-oriented way or a functional way.



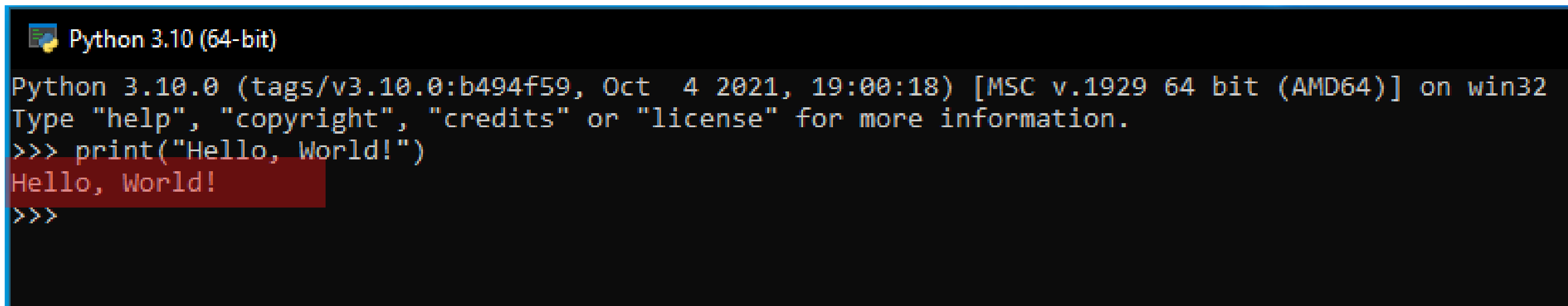
PYTHON SYNTAX COMPARED TO OTHER PROGRAMMING LANGUAGES

- Python was designed for readability, and has some similarities to the **English language** with influence from **mathematics**.
- Python uses **new lines** to complete a command, as opposed to other programming languages which often use **semicolons** or **parentheses**.
- Python relies on **indentation**, using **whitespace**, to **define scope**; such as the scope of **loops**, **functions** and **classes**. Other programming languages often use **curly-brackets** for this purpose.

EXAMPLE

```
print("Hello, World!")
```

OUTPUT

A screenshot of a Python 3.10 (64-bit) terminal window. The window title is "Python 3.10 (64-bit)". The terminal text shows the Python version and build information: "Python 3.10.0 (tags/v3.10.0:b494f59, Oct 4 2021, 19:00:18) [MSC v.1929 64 bit (AMD64)] on win32". It then prompts the user to type "help", "copyright", "credits", or "license" for more information. The user enters ">>> print('Hello, World!')", and the terminal outputs "Hello, World!". The prompt ">>>" is shown again on the next line.

```
Python 3.10 (64-bit)  
Python 3.10.0 (tags/v3.10.0:b494f59, Oct 4 2021, 19:00:18) [MSC v.1929 64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license" for more information.  
>>> print("Hello, World!")  
Hello, World!  
>>>
```

PYTHON INSTALL

- Many PCs and Macs will have python already installed.
- To check if you have python installed on a Windows PC, search in the start bar for Python or run the following on the Command Line (cmd.exe):

OUTPUT

```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.19041.1348]
(c) Microsoft Corporation. All rights reserved.

C:\Users\USEP TAGUM>python --version
Python 3.10.0

C:\Users\USEP TAGUM>
```

PYTHON QUICKSTART

- Python is an interpreted programming language, this means that as a developer you write **Python** (.py) files in a text editor and then put those files into the **python interpreter** to be executed.
- The way to run a python file is like this on the command line:

```
C:\Users\Your Name>python helloworld.py
```

- Simple as that. Save your file. Open your command line, navigate to the directory where you saved your file, and run:

```
C:\Users\Your Name>python helloworld.py
```

THE OUTPUT SHOULD READ:

```
Hello, World!
```

PYTHON SYNTAX

PYTHON INDENTATION

- **Indentation** refers to the **spaces** at the beginning of a code line.
- Where in other programming languages the indentation in code is for readability only, the indentation in Python is very important.
- Python uses **indentation** to indicate a block of code.

EXAMPLE

```
if 5 > 2:  
    print("Five is greater than two!")
```

```
if 5 > 2:  
    print("Five is greater than two!")  
if 5 > 2:  
    print("Five is greater than two!")
```

The number of spaces is up to you as a programmer, but it has to be at least one.

PYTHON COMMENTS

- Comments can be used to explain Python code.
- Comments can be used to make the code more readable.
- Comments can be used to prevent execution when testing code.

CREATING A COMMENT

Comments starts with a #, and Python will ignore them:

EXAMPLE

```
#This is a comment  
print("Hello, World!")
```

EXAMPLE

```
print("Hello, World!") #This is a comment
```

Comments can be placed at the end of a line, and Python will ignore the rest of the line:

PYTHON VARIABLES

Variables are containers for storing data values.

CREATING VARIABLES

- Python has no command for declaring a variable.
- A variable is created the moment you first assign a value to it.

EXAMPLE

```
x = 5  
y = "John"  
print(x)  
print(y)
```

OUTPUT

```
5  
John
```

PYTHON VARIABLES

Variables do not need to be declared with any particular type, and can even change type after they have been set.

EXAMPLE

```
x = 4           # x is of type int  
x = "Sally"     # x is now of type str  
print(x)
```

OUTPUT



```
Sally
```

CASTING

If you want to specify the data type of a variable, this can be done with casting.

EXAMPLE

```
x = str(3)  # x will be '3'
```

```
y = int(3)  # y will be 3
```

```
z = float(3) # z will be 3.0
```

OUTPUT

```
3
```

```
3
```

```
3.0
```

GET THE TYPE

You can get the data type of a variable with the `type()` function.

EXAMPLE

```
x = 5  
y = "John"  
print(type(x))  
print(type(y))
```

OUTPUT

```
<class 'int'>  
<class 'str'>
```

SINGLE OR DOUBLE QUOTES?

String variables can be declared either by using single or double quotes:

EXAMPLE

```
x = "John"
```

is the same as

```
x = 'John'
```

OUTPUT

```
John
```

```
John
```

CASE-SENSITIVE

Variable names are case-sensitive.

EXAMPLE

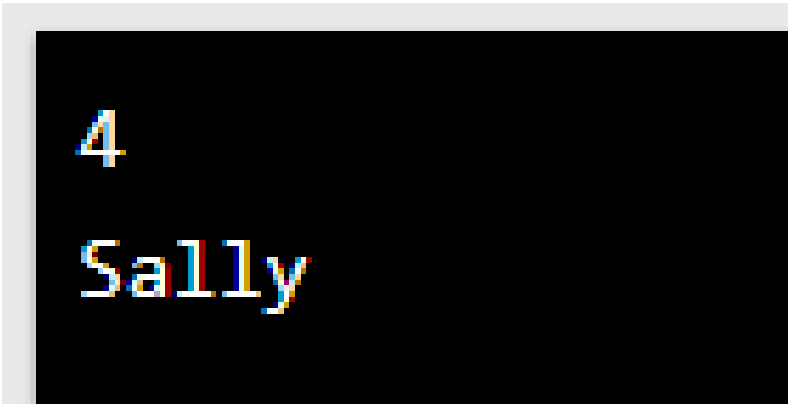
This will create two variables:

```
a = 4
```

```
A = "Sally"
```

```
#A will not overwrite a
```

OUTPUT

A terminal window with a black background and a light gray border. It displays the output of a program. The first line shows the number '4' in a light blue color. The second line shows the string 'Sally' in a light blue color. The text is rendered in a monospaced font with a slight shadow effect.

```
4
```

```
Sally
```

PYTHON - VARIABLE NAMES

- A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume).

Rules for Python variables:

- A variable name must start with a letter or the **underscore** character
- A variable name cannot start with a **number**
- A variable name can only contain **alpha-numeric characters** and **underscores** (A-z, 0-9, and _)
- Variable names are **case-sensitive** (age, Age and AGE are three different variables)

EXAMPLE

```
myvar = "John"  
my_var = "John"  
_my_var = "John"  
myVar = "John"  
MYVAR = "John"  
myvar2 = "John"
```

OUTPUT

```
John  
John  
John  
John  
John  
John
```

ILLEGAL VARIABLE NAMES:

EXAMPLE

2myvar = "John"

my-var = "John"

my var = "John"

Remember that variable names are case-sensitive

MULTI WORDS VARIABLE NAMES

- Variable names with more than one word can be difficult to read.
- There are several techniques you can use to make them more readable:

CAMEL CASE

Each word, except the first, starts with a capital letter:

```
myVariableName = "John"
```

PASCAL CASE

Each word starts with a capital letter:

```
MyVariableName = "John"
```

SNAKE CASE

Each word is separated by an underscore character:

```
my_variable_name = "John"
```

PYTHON VARIABLES - ASSIGN MULTIPLE VALUES

Python allows you to assign values to multiple variables in one line:

EXAMPLE

```
x, y, z = "Orange", "Banana", "Cherry"  
print(x)  
print(y)  
print(z)
```

OUTPUT

```
Orange  
Banana  
Cherry
```

ONE VALUE TO MULTIPLE VARIABLES

And you can assign the same value to multiple variables in one line:

EXAMPLE

```
x = y = z = "Orange"  
print(x)  
print(y)  
print(z)
```

OUTPUT

```
Orange  
Orange  
Orange
```

UNPACK A COLLECTION

If you have a collection of values in a list, tuple etc. Python allows you extract the values into variables. This is called **unpacking**.

EXAMPLE

```
fruits = ["apple", "banana", "cherry"]  
x, y, z = fruits  
print(x)  
print(y)  
print(z)
```

OUTPUT

```
apple  
banana  
cherry
```

PYTHON - OUTPUT VARIABLES

The Python **print** statement is often used to output variables.
To combine both text and a variable, Python uses the + character:

EXAMPLE

```
x = "awesome"  
print("Python is " + x)
```

OUTPUT

```
Python is awesome
```

PYTHON - OUTPUT VARIABLES

You can also use the + character to add a variable to another variable:

EXAMPLE

```
x = "Python is "  
y = "awesome"  
z = x + y  
print(z)
```

OUTPUT

```
Python is awesome
```

PYTHON - OUTPUT VARIABLES

For numbers, the + character works as a mathematical operator:

EXAMPLE

```
x = 5  
y = 10  
print(x + y)
```

OUTPUT

```
15
```

If you try to combine a string and a number, Python will give you an error:

EXAMPLE

```
x = 5  
y = 10  
print(x + y)
```

OUTPUT

```
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

GETTING THE DATA TYPE

You can get the data type of any object by using the `type()` function:

EXAMPLE

Print the data type of the variable `x`:

```
x = 5
```

```
print(type(x))
```

PYTHON NUMBERS

Int

Int, or integer, is a whole number, positive or negative, without decimals, of unlimited length.

EXAMPLE

```
x = 1
```

```
y = 35656222554887711
```

```
z = -3255522
```

```
print(type(x))
```

```
print(type(y))
```

```
print(type(z))
```

OUTPUT

```
<class 'int'>
```

```
<class 'int'>
```

```
<class 'int'>
```

PYTHON NUMBERS

Float

Float, or "floating point number" is a number, positive or negative, containing one or more decimals.

EXAMPLE

```
x = 1.10
```

```
y = 1.0
```

```
z = -35.59
```

```
print(type(x))
```

```
print(type(y))
```

```
print(type(z))
```

OUTPUT

```
<class 'float'>  
<class 'float'>  
<class 'float'>
```

PYTHON NUMBERS

Type Conversion

You can convert from one type to another with the `int()`, `float()`, and `complex()` methods:

EXAMPLE

```
x = 1 # int
y = 2.8 # float
z = 1j # complex

#convert from int to float:
a = float(x)

#convert from float to int:
b = int(y)

#convert from int to complex:
c = complex(x)

print(a)
print(b)
print(c)

print(type(a))
print(type(b))
print(type(c))
```

OUTPUT

```
1.0
2
(1+0j)
<class 'float'>
<class 'int'>
<class 'complex'>
```

PYTHON CASTING

Specify a Variable Type

There may be times when you want to specify a type on to a variable. This can be done with **casting**. Python is an object-orientated language, and as such it uses classes to define data types, including its primitive types.

Casting in python is therefore done using constructor functions:

- **int()** - constructs an integer number from an integer literal, a float literal (by removing all decimals), or a string literal (providing the string represents a whole number)
- **float()** - constructs a float number from an integer literal, a float literal or a string literal (providing the string represents a float or an integer)
- **str()** - constructs a string from a wide variety of data types, including strings, integer literals and float literals

PYTHON CASTING

EXAMPLE

Integers:

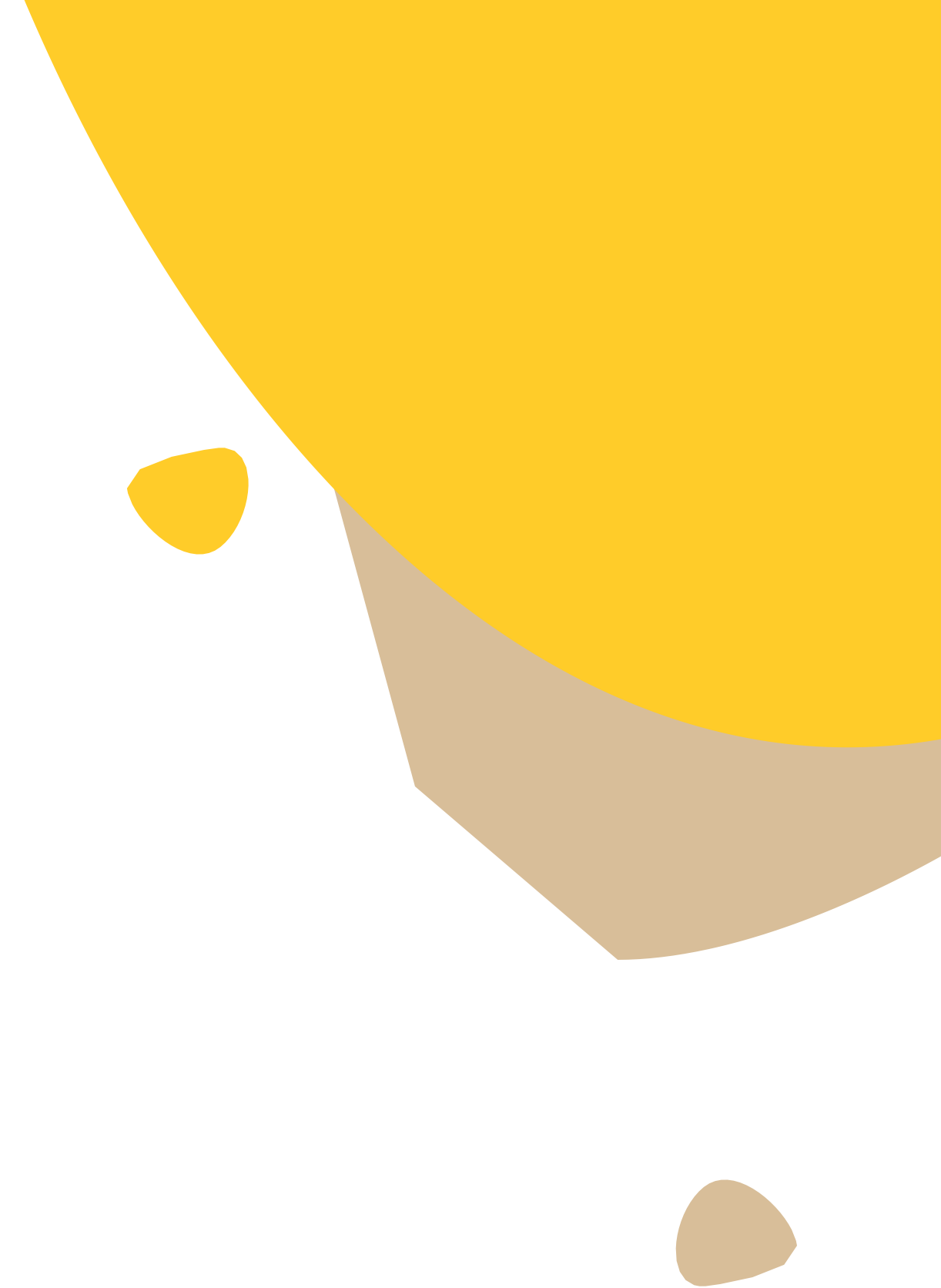
```
x = int(1) # x will be 1  
y = int(2.8) # y will be 2  
z = int("3") # z will be 3
```

Floats:

```
x = float(1) # x will be 1.0  
y = float(2.8) # y will be 2.8  
z = float("3") # z will be 3.0  
w = float("4.2") # w will be 4.2
```

Strings:

```
x = str("s1") # x will be 's1'  
y = str(2) # y will be '2'  
z = str(3.0) # z will be '3.0'
```



PYTHON STRINGS

- Strings in python are surrounded by either single quotation marks, or double quotation marks.
- 'hello' is the same as "hello".
- You can display a string literal with the print() function:

EXAMPLE

```
print("Hello")  
print('Hello')
```

ASSIGN STRING TO A VARIABLE

Assigning a string to a variable is done with the variable name followed by an equal sign and the string:

EXAMPLE

```
a = "Hello"  
print(a)
```

MULTILINE STRINGS

- You can assign a multiline string to a variable by using three quotes:

EXAMPLE

```
a = """Lorem ipsum dolor sit amet,  
consectetur adipiscing elit,  
sed do eiusmod tempor incididunt  
ut labore et dolore magna aliqua."""  
print(a)
```

Or three single quotes:

```
a = '''Lorem ipsum dolor sit amet,  
consectetur adipiscing elit,  
sed do eiusmod tempor incididunt  
ut labore et dolore magna aliqua.'''  
print(a)
```

CHECK STRING

- To check if a certain phrase or character is present in a string, we can use the keyword `in`.

EXAMPLE

Check if "free" is present in the following text:

```
txt = "The best things in life are free!"  
print("free" in txt)
```

OUTPUT



```
True
```

PYTHON - MODIFY STRINGS

- Python has a set of built-in methods that you can use on strings.

UPPER CASE

EXAMPLE

The upper() method returns the string in upper case:

```
a = "Hello, World!"  
print(a.upper())
```

OUTPUT

```
HELLO, WORLD!
```

PYTHON - MODIFY STRINGS

UPPER CASE

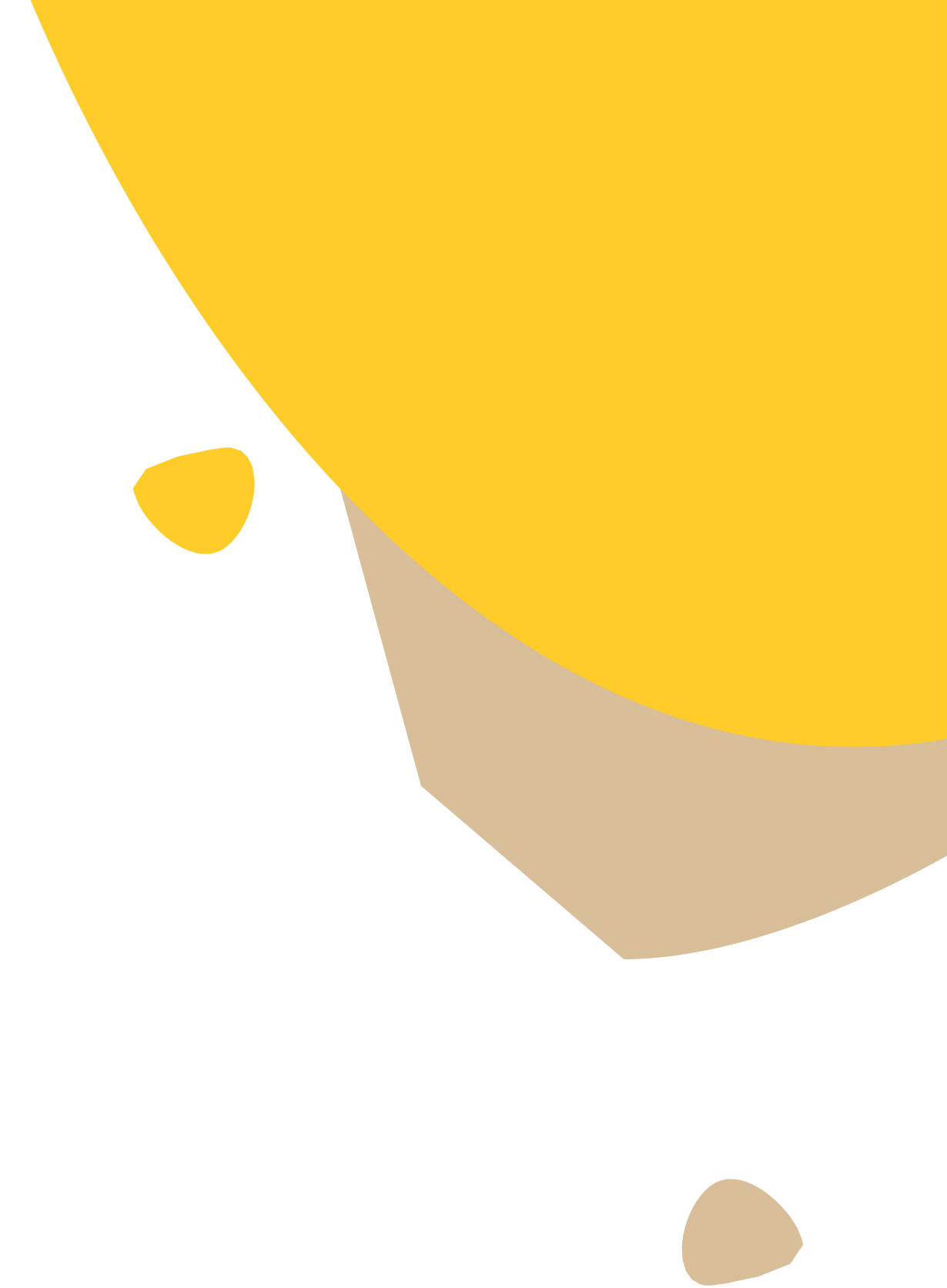
EXAMPLE

The lower() method returns the string in lower case:

```
a = "Hello, World!"  
print(a.lower())
```

OUTPUT

```
hello, world!
```



PYTHON - STRING CONCATENATION

To concatenate, or combine, two strings you can use the + operator.

EXAMPLE

Merge variable a with variable b into variable c:

```
a = "Hello"  
b = "World"  
c = a + b  
print(c)
```

OUTPUT



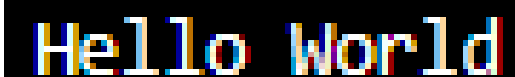
```
HelloWorld
```

EXAMPLE

To add a space between them, add a " ":

```
a = "Hello"  
b = "World"  
c = a + " " + b  
print(c)
```

OUTPUT



```
Hello World
```

PYTHON - STRING METHODS

Python has a set of built-in methods that you can use on strings.

Method	Description
<code>capitalize()</code>	Converts the first character to upper case
<code>casefold()</code>	Converts string into lower case
<code>center()</code>	Returns a centered string
<code>count()</code>	Returns the number of times a specified value occurs in a string
<code>encode()</code>	Returns an encoded version of the string
<code>endswith()</code>	Returns true if the string ends with the specified value
<code>expandtabs()</code>	Sets the tab size of the string
<code>find()</code>	Searches the string for a specified value and returns the position of where it was found
<code>format()</code>	Formats specified values in a string
<code>format_map()</code>	Formats specified values in a string
<code>index()</code>	Searches the string for a specified value and returns the position of where it was found
<code>isalnum()</code>	Returns True if all characters in the string are alphanumeric
<code>isalpha()</code>	Returns True if all characters in the string are in the alphabet
<code>isdecimal()</code>	Returns True if all characters in the string are decimals

<code>isdigit()</code>	Returns True if all characters in the string are digits
<code>isidentifier()</code>	Returns True if the string is an identifier
<code>islower()</code>	Returns True if all characters in the string are lower case
<code>isnumeric()</code>	Returns True if all characters in the string are numeric
<code>isprintable()</code>	Returns True if all characters in the string are printable
<code>isspace()</code>	Returns True if all characters in the string are whitespaces
<code>istitle()</code>	Returns True if the string follows the rules of a title
<code>isupper()</code>	Returns True if all characters in the string are upper case
<code>join()</code>	Joins the elements of an iterable to the end of the string
<code>ljust()</code>	Returns a left justified version of the string
<code>lower()</code>	Converts a string into lower case
<code>lstrip()</code>	Returns a left trim version of the string
<code>maketrans()</code>	Returns a translation table to be used in translations
<code>partition()</code>	Returns a tuple where the string is parted into three parts
<code>replace()</code>	Returns a string where a specified value is replaced with a specified value
<code>rfind()</code>	Searches the string for a specified value and returns the last position of where it was found
<code>rindex()</code>	Searches the string for a specified value and returns the last position of where it was found

PYTHON - STRING METHODS

Python has a set of built-in methods that you can use on strings.

<code>rjust()</code>	Returns a right justified version of the string
<code>rpartition()</code>	Returns a tuple where the string is parted into three parts
<code>rsplit()</code>	Splits the string at the specified separator, and returns a list
<code>rstrip()</code>	Returns a right trim version of the string
<code>split()</code>	Splits the string at the specified separator, and returns a list
<code>splitlines()</code>	Splits the string at line breaks and returns a list
<code>startswith()</code>	Returns true if the string starts with the specified value
<code>strip()</code>	Returns a trimmed version of the string
<code>swapcase()</code>	Swaps cases, lower case becomes upper case and vice versa
<code>title()</code>	Converts the first character of each word to upper case
<code>translate()</code>	Returns a translated string
<code>upper()</code>	Converts a string into upper case
<code>zfill()</code>	Fills the string with a specified number of 0 values at the beginning

EXERCISES

1. Insert the missing indentation to make the code correct:

```
if 5 > 2:  
    print("Five is greater than two!")
```

2. Comments in Python are written with a special character, which one?

```
# This is a comment
```

3. Create a variable named carname and assign the value Volvo to it.

4. Create a variable called z, assign x + y to it, and display the result.

```
x = 5  
y = 10  
z = x + y  
print(z)
```

EXERCISES

5. The following code example would print the data type of x, what data type would that be?

```
x = 20.5  
print(type(x))
```

6. The following code example would print the data type of x, what data type would that be?

```
x = ["apple", "banana", "cherry"]  
print(type(x))
```

7. Insert the correct syntax to convert x into a floating point number.

```
x = 5  
x =  (x)
```

8. Convert the value of txt to lower case.

```
txt = "Hello World"  
txt = 
```